

System Architecture

Feature Descriptions

Shared Features

Site-Admin

Restaurant-Owner

Customer

Customers can:

Browse Restaurants

Frontend

Navigation & Network

Modules

Backend

Domains

Middleware

Routing

Repositories

Site Manager

Restaurant Owner

Customer

Database

User

Restaurant

Customer (Restaurant data)

Order

Shared Components and Backend Services

Generic Request Flow

User Registration & Authentication

Login

Profile Management

Responsive UI

Error Handling

Async Handler

Navigation & Routing

Distance Simulation

Extra Tasks

Real-Time Order Tracking

Key features

Implementation Details

Key Features Summary

Setup Instructions

Backend Setup

Frontend Setup

Environment Variables

System Architecture

The repository-root contains three folders containing the Angular frontend (`client`), the database migrations (`db`) and the backend (`server`). The Customer uses the frontend to communicate with the server. Data on the database is exclusively accessed via the server.

Topic	Responsible Person
Site Manager	Armin
Restaurant Owner	Armin
Customer (User)	Miriam

Feature Descriptions

Shared Features

During user registration users

- have to provide required data and
- can choose which role they want to have.
 - additional information for restaurants has to be provided if “Restaurant Owner” is chosen

Profile management allows users to

- update their personal data,
- manage multiple addresses and
- changing their password.

Site-Admin

The `site-admin@freelivery.com` user provides all testdata for the given functionality. It can:

- get an overview of some core statistics
- see all active restaurants and set a service-fee for each
- see pending erstaurant requests and approve/decline them
- check user details and suspend them

Restaurant-Owner

The `restaurant-owner@freelivery.com` user provides all testdata for the given functionality. It can:

- manage the menu by
 - creating and editing categories including
 - a category name and
 - a category description
 - creating and editing dishes including
 - an image,
 - a dish name,
 - optional description and
 - price.
 - setting dishes as unavailable
 - deleting dishes and categories
- manage orders by
 - accepting or rejecting them and
 - updating their status throughout the process.

- manage their opening hours
- get basic analytics for their restaurant like
 - weekly orders and revenue,
 - a daily breakdown of the last weeks
 - identifying most ordered dishes.

Customer

The Customer module provides an overview of the food delivery experience from the customer’s side. The module is what users see, use and interact with. Users can browse restaurants, order food, choose their preferred paying method and track their order in real time.

Customers can:

Browse Restaurants

- Check all available restaurants in the area
- Filter restaurants by cuisine type
- Search for restaurants by name
- Filter restaurants by highest rated && quickest delivery time

Place an order

- Placing an order by choosing a menu item
- Adding menu items to cart

Cart view

- View the cart and its items
- Update the cart by removing and adding new items and items quantity
- Clearing the cart
- Applying the promo code ‘PROMO26’
- Proceeding to checkout

Choose a payment method

- The user can choose a preferred payment method:
 - by **cash** at order arrival
 - by **card** directly in the website by giving the card details

Track orders in Real-Time

- The user can see the order tracking **live** through:
 - **status updates** - get notified when the order is being prepared, ready, delivering and delivered
 - **chat with restaurant** - viewing the order updates, live, through the chat by the restaurant and messaging the restaurant
 - **delivery time estimates** - know when to expect the order

Review the Experience after Order Delivery

- The user can review the service provided by selecting a preferred review emoji and by writing a comment

Frontend

```
.
├── src
│   ├── app
│   ├── commons
│   │   ├── guards
│   │   ├── interceptors
│   │   ├── model
│   │   ├── pipes
│   │   └── services
│   ├── layout
│   └── modules
│       ├── customer
│       └── profile
```

Navigation & Network

app.routes.ts

- defines routes to
 - central functions
 - login, signup, profile
 - lazy loaded modules
 - customer
 - restaurant
 - site-manager
 - profile
- where necessary, routes are protected via `canActivate` using `authGuard`
 - module-specific routes are defined in their respective routes-file

auth.guard.ts

- uses the `authentication.service.ts` to verify whether a user is logged in and
- verifies that the logged in user possesses the role required for a given route

authenticator.interceptor.ts

- clones request (conceptually immutable) and sets authorization header

serverError.interceptor.ts

- taps into responses and verifies their format and
- catches any `HttpErrors` and surfaces them in a dismissible snackbar

app.config.ts

- `provideHttpClient(withInterceptors([...]))` provides the interceptors as `HttpHandler` to the `HttpClient` in the given order

Modules

Site-manager

- all functionality is contained in the `site-manager-home`
- `site-manager.models.ts` contains site-manager specific models for
 - pending restaurant registrations
 - user management and
 - dashboard statistics

Restaurant-owner

- `restaurant-owner.routes.ts`
 - defines lazy-loded routes used in `app.routes.ts`
- `restaurant-owner.service.ts`
 - contains general DTOs and DTOs for creating/updating given entities
 - provides methods used solely by the `restaurant-owner` -module
 - all calls to `/api/restaurant-owner` require the `restaurant-owner` role (see backend section)
 - manipulating restaurant-owner specific entities
- other folders in the module
 - contain form-elements and other reusable components

Customer

Path	Component	Description
/customer	CustomerHomeComponent	Main dashboard
/customer/home	CustomerHomeComponent	Alternative dashboard path
/customer/restaurants	RestaurantBrowsingComponent	Browse all restaurants
/customer/menu/:id	ViewMenuComponent	View restaurant menu
/customer/cart	CartPageComponent	Shopping cart
/customer/checkout	CheckoutComponent	Place order
/customer/tracking/:id	OrderTrackingComponent	Track order status
/customer/feedback	FeedbackComponent	Submit feedback

CustomerService

- Main HTTP customer for customer-related API calls
- Methods used:
 - getRestaurants(): get all active restaurants
 - getRestaurantById(id): get restaurant details
 - getRestaurantMenu(id): get menu items
 - getRestairantCategories(id): get menu categories
 - getCartCount(): get total cart items

RestaurantBrowsingService

- Delegated to CustomerService for data fetching
- getRestaurantWithMenu(): paralled data loading
- Promise.all(): API calls

CartService

- Manage cart state in localStorage
- Methods used:
 - addToCart(): add items with restaurant validation
 - removeFromCart(): remove item by dishId
 - changeQuantity(): update item quantity
 - clearCart(): empty cart
 - getTotal(): calculate final price with delivery fee

WebSocketService

- Simulates real- time Websocket connection for order tracking
- Emits order status updates through RxJS
- Calculates delivery time based on grid distance
- Simulates restaurant chat messaes
- Methods used:
 - connect(): initialize Websocket connection
 - disconnect(): disconnect Websocket connection
 - sendMessage(): send chat message
 - calculateDistance(): Calculating distance based on grid coordinates
 - simulateStatusUpdates(): simulates automatic status changes

customer.service.ts

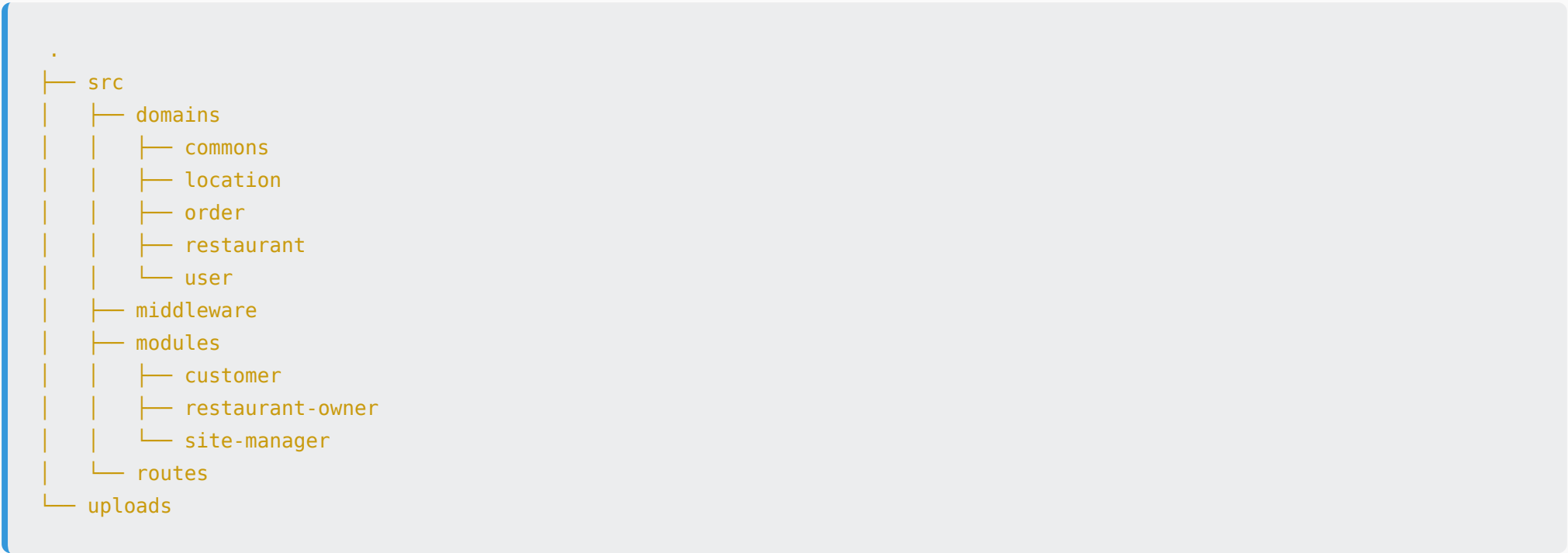
- Business logic for customer operations

- Order data validation before creation
- Manages order status transitions

customer.repository.ts

- Database queries for customer data
- Fetching restaurants with availability status
- Menu items retrival by restaurant
- Create and update orders

Backend



Domains

- **commons** contains general-purpose classes and services shared across different modules
 - **abstract-repository** defines an abstract singleton class which manages the database connection
 - **errors**
 - provides the **expressErrorHandler** which converts any generic errors into **AppError** s
 - generate HTTP error responses from **AppError**
 - provides pre-defined errors which can be thrown in the code together with a error-message for the client
 - **repository-registry** provides methods to get the corresponding repositories
 - **statistics-repository** + **stastics-service** for shared statistics
- **location** , **order** , **restaurant** , **user** group shared and domain-specific models and functionalities like repositories and services

Middleware

- **middleware** defines all middleware for ...
 - authentication
 - logging
 - file-upload
 - not-found handling

Routing

Routing is split into multiple parts:

- some generally available entities provide routes for authenticated users
 - **restaurant** domain exposes all categories and dishes for a restaurant and all active restaurants
 - authentication is verified via a **requireAuth** middleware
- anything requiring a specific **role** is only available to users with that given role
 - this is realised using a **requireRole** middleware
 - routes are defined in that modules's folder

index.ts

- endpoint for HTTP requests
- `/uploads` served statically for image hosting
- *public* endpoints:
 - `/api/auth` to authenticate users
- *authenticated* endpoints: any logged in user can access them
 - `/api/addresses`
 - CRUD endpoints for addresses
 - `/api/restaurants`
 - get active restaurants and
 - their categories/dishes
 - `/api/profile`
 - profile management (updating password & address)
 - restaurant management
- *authenticated & restricted* endpoints: only users with given role can access
 - `/api/site-manager`
 - displaying all users
 - getting site statistics
 - suspending users
 - approving and rejecting restaurant
 - updating service-fee for restaurants
 - `/api/restaurant-owner`
 - CRUD endpoints for managing the menu
 - accessing orders and updating their status
 - managing opening hours
 - accessing analytics

Repositories

All repositories extend `Repository<T>` from `abstract-repository.ts` :

- manages a **static singleton** `Pool` for database connections (lazily initialized)
- provides generic CRUD helpers: `findById` , `findAll` , `create` , `update` , `delete`
- wraps errors into `AppError` via `toAppError()`

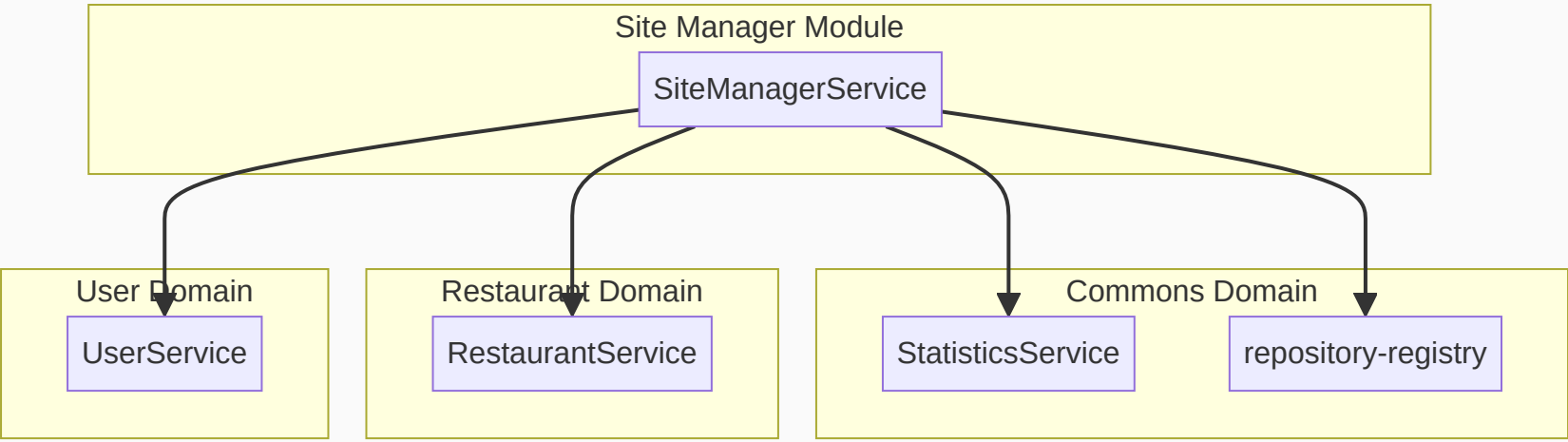
`repository-registry.ts` provides getter functions for each repository:

- `getUserRepository()` , `getAddressRepository()` , `getRestaurantRepository()` , etc.
- lazily instantiates and caches singletons
- ensures all services share the same repository instances

Site Manager

The site manager backend module provides only basic functionality like ...

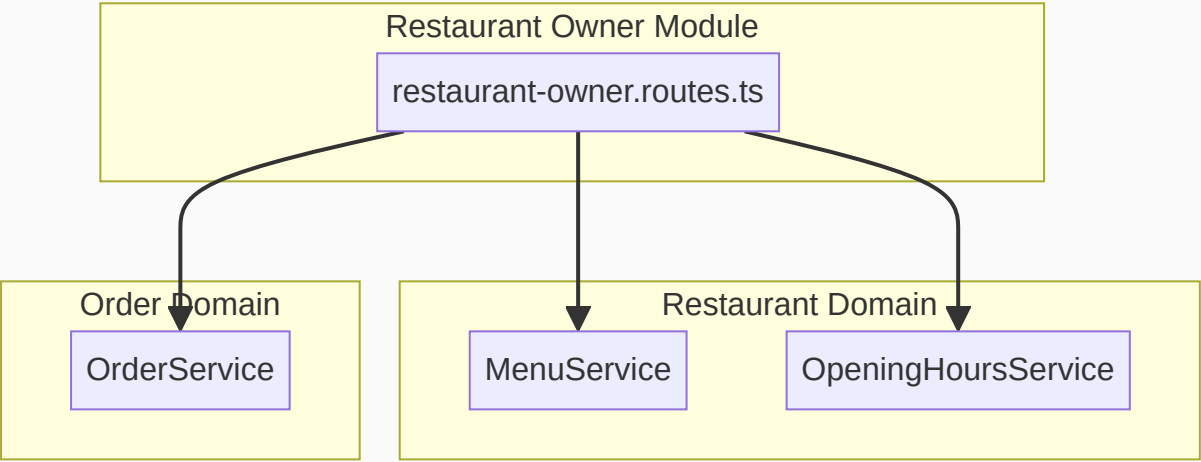
- suspending users,
- approving and rejecting restaurants and
- setting a service fee per restaurant.



Restaurant Owner

The restaurant owner provides feature-complete functionality for restaurant owners and provides the following endpoints through the `restaurant-owner.routes.ts` :

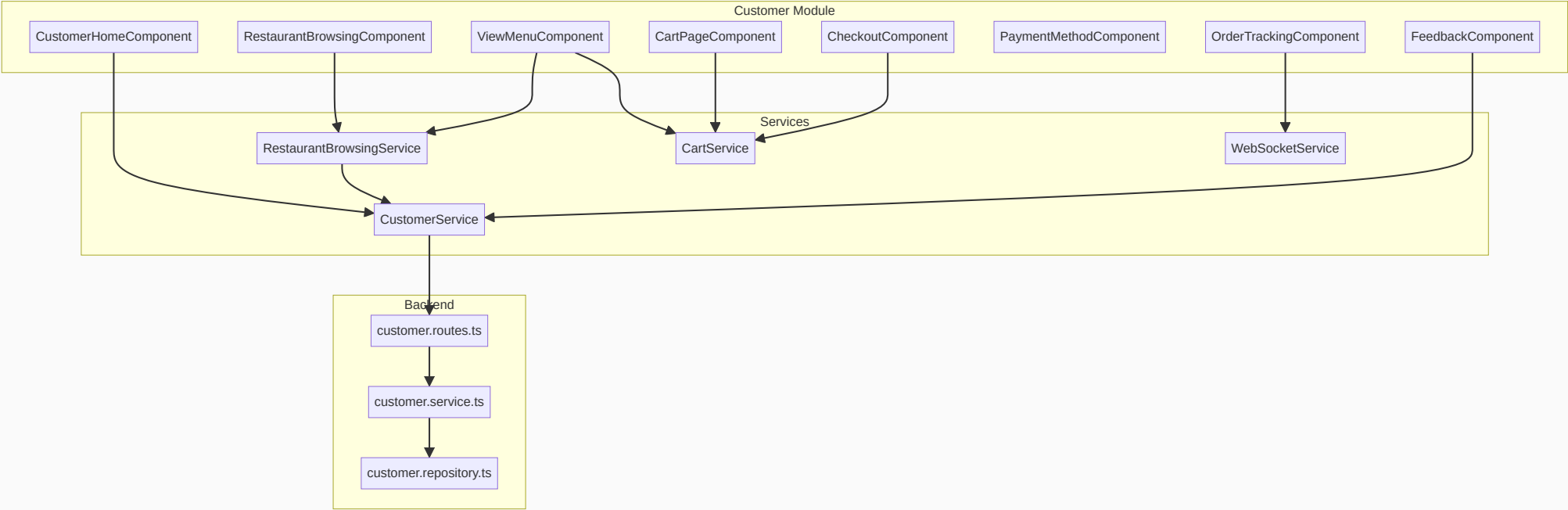
- get the current signed in user’s owned restaurant
- CRUD endpoints for
 - dishes and categories
 - opening hours
- restaurant-owner specific order routes
 - getting all orders
 - updating the status
- analytics for the resautarnt



Customer

The customer provides features like:

- Database queries for customer data
- Fetches restaurants with availability status
- Menu items retrieval by restaurant
- Creates and updates orders



Database

Note: the datatypes given in the diagram are the ones used in the backend; **DECIMAL** and **TIME** types are parsed as strings.

User

Table descriptions:

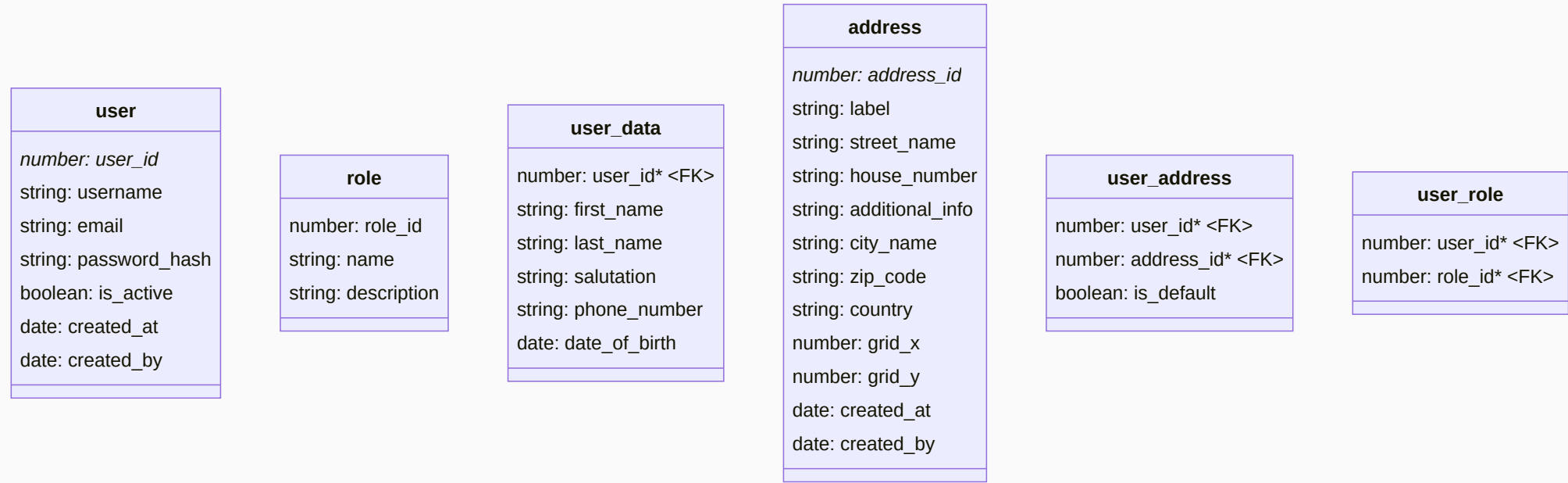
- user** contains the core data for authentication
- user_data** extends the user data by central fields
- address** stores address-details
- role** contains all available roles
- user_address** assigns addresses to users
- user_role** assigns roles to users

available users in the testdatamigration are:

Username	Email	Password	Role(s)
customer	customer@freelivery.com	customer	customer
restaurant-owner	restaurant-owner@freelivery.com	restaurant	restaurant_owner
site-admin	site-admin@freelivery.com	site-admin	admin
deus	deus@freelivery.com	deus	admin, restaurant_owner, customer
alice	alice@example.com	passhash1	customer
bob	bob@example.com	passhash2	restaurant_owner

Seed data for the site-manager contains

- active and pending restaurant requests for approval/rejection
- list of all users



Restaurant

Table descriptions:

- restaurant** stores restaurant details, fees, and links to owner and address
- category** groups dishes within a restaurant's menu
- dish** contains individual menu items with pricing and availability
- opening_hours** defines business hours per day of week

Seed data:

- user **restaurant-owner** has data for ...
 - a complete menu (excluding any pictures)
 - existing orders in all available states
 - opening hours

restaurant	category	dish	opening_hours
<i>number: restaurant_id</i> string: name status: restaurant_status string: description cuisine_type: cuisine_type string: contact_email string: contact_phone number: address_id <FK> number: owner_user_id <FK> string: service_fee_percent string: min_order_amount number: max_delivery_distance date: created_at date: updated_at	<i>number: category_id</i> number: restaurant_id <FK> string: name string: description date: created_at date: updated_at	<i>number: dish_id</i> number: restaurant_id <FK> number: category_id <FK> string: name string: description string: price string: image_url boolean: is_available date: created_at date: updated_at	<i>number: opening_hours_id</i> number: restaurant_id <FK> number: day_of_week string: open_time string: close_time date: created_at date: updated_at

Customer (Restaurant data)

Seed data:

- user **customer** has data for testing the customer module including:
 - three additional restaurants for browsing and ordering:
 - Pizzeria Piccola Napoli (Italian cuisine)
 - Chicago Burger Shop (American cuisine)
 - Shanghai Palace (Japanese cuisine)
 - complete menus with categories and dishes for each restaurant
 - grid coordinates for delivery distance calculation

Restaurant Details:

Restaurant	Cuisine	Address	Grid Location	Min Order	Max Distance	Service Fee
Pizzeria Piccola Napoli	ITALIAN	Neutilusweg 25, Klagenfurt	(-1, 3)	€10.00	25 units	2.00%
Chicago Burger Shop	AMERICAN	Friedensgasse 48, Klagenfurt	(3, -2)	€10.00	50 units	3.00%
Shanghai Palace	JAPANESE	Kramergasse 2, Klagenfurt	(2, 5)	€12.00	40 units	3.00%

Order

Table descriptions:

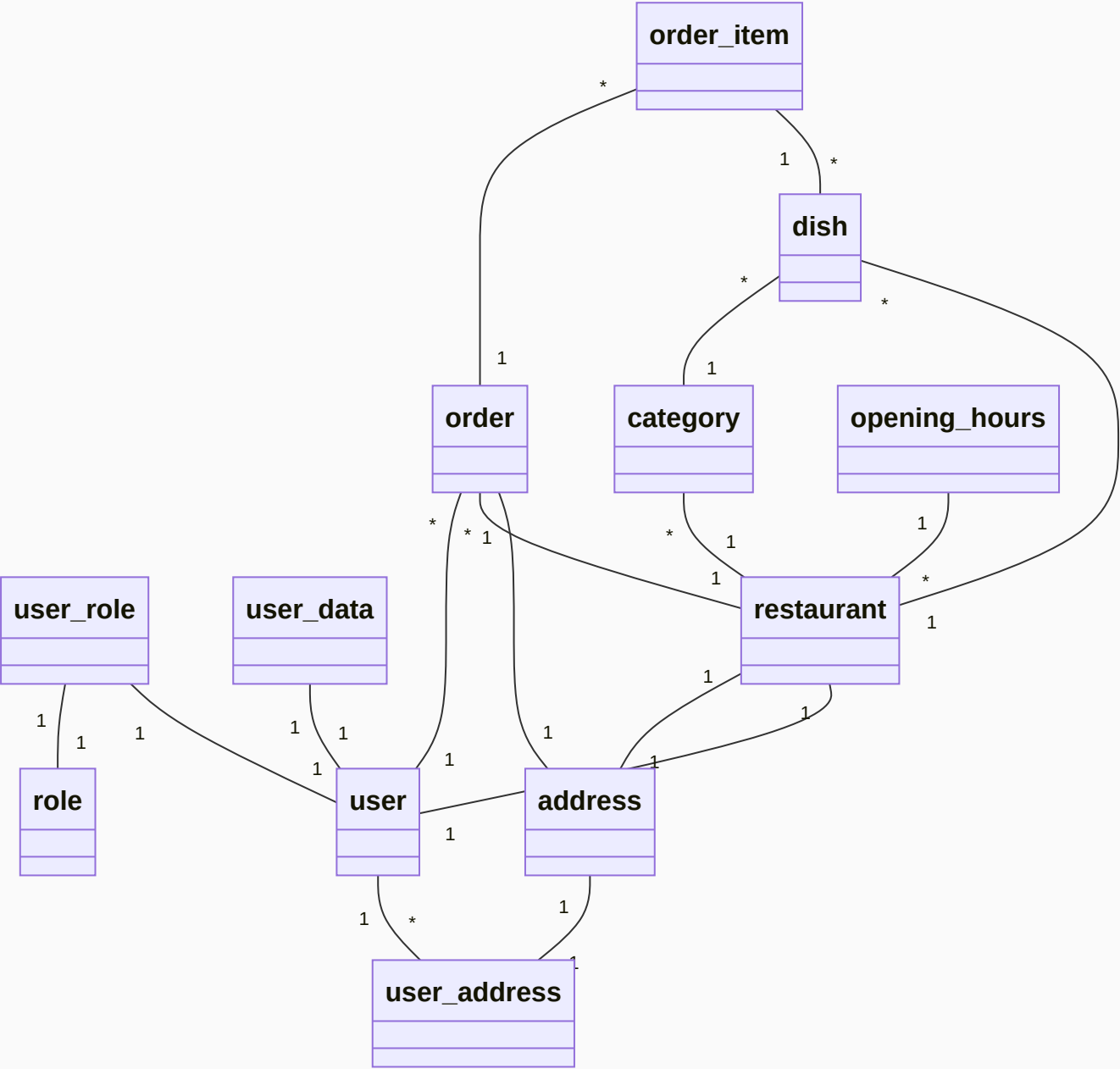
- **order** tracks customer orders with pricing, status, and delivery info
- **order_item** stores individual items within an order with snapshot data for historic information

Seed data:

- contains historic orders for **restaurant-owner@freelivery.com**

order
<i>number: order_id</i>
number: customer_user_id <FK>
number: restaurant_id <FK>
number: delivery_address_id <FK>
order_status: status
string: subtotal_amount
string: service_fee_amount
string: discount_amount
string: total_amount
payment_method: payment_method
date: estimated_delivery_time
date: delivered_at
date: created_at
date: updated_at

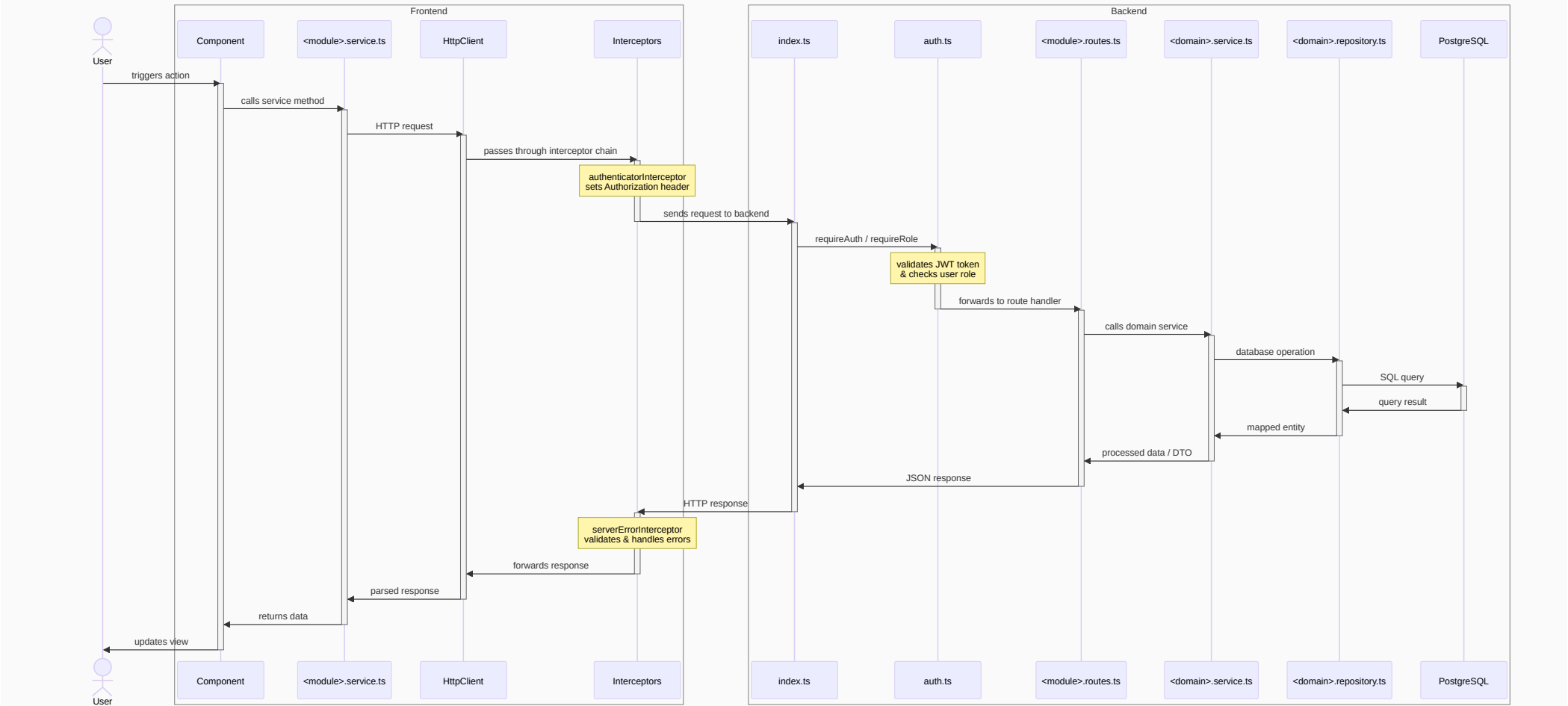
order_item
<i>number: order_item_id</i>
number: order_id <FK>
number: dish_id <FK>
string: dish_name_snapshot
string: unit_price
number: quantity
date: created_at



Shared Components and Backend Services

Generic Request Flow

Note: this is the idealised architecture we aimed for. However, not all parts of the example below are implemented in this exact manner.



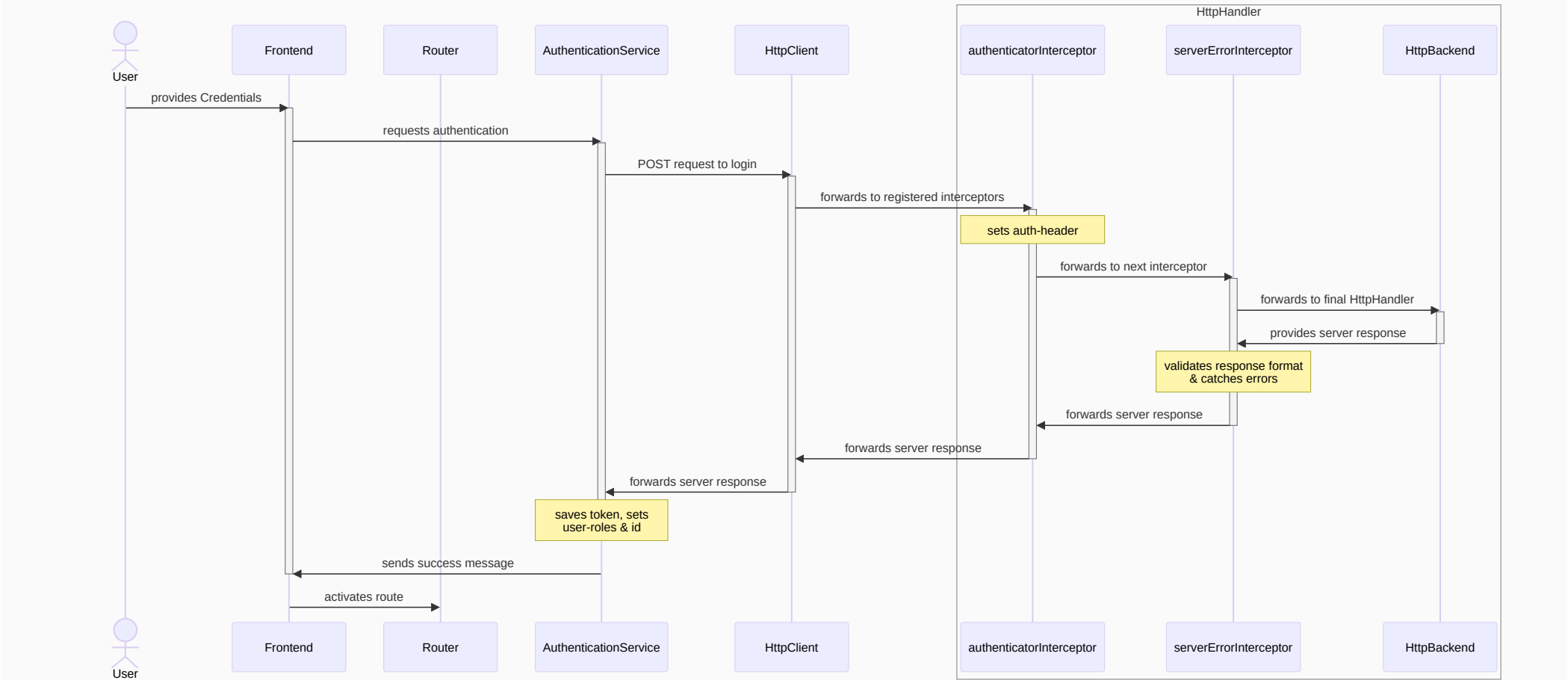
User Registration & Authentication

implemented by: Armin Lachini

Related code-parts:

- `server/src/routes/auth.routes.ts`
- `server/src/middleware/auth.ts`
- `client/src/commons/interceptors/authenticator.interceptor.ts`
- `client/src/commons/guards/auth.guard.ts`
- `client/src/commons/services/authentication.service.ts`
- `client/src/layout/login/*`
- `client/src/layout/signup/*`

Login



Profile Management

implemented by: Armin Lachini

Related code-parts:

- `client/src/modules/profile/*`

- Angular components for viewing and editing user profile
 - delegates address operations to `AddressService`
- `server/src/routes/profile.routes.ts`
 - GET/PUT `/api/profile` for user profile data
 - PUT `/api/profile/password` for password changes
 - GET/PUT `/api/profile/restaurant` for restaurant owners

Responsive UI

implemented by: Armin Lachini

All components use CSS media queries with a 640px breakpoint:

- `@media (max-width: 640px)` for mobile-specific styles
- navbar collapses into a hamburger menu
- form fields stack vertically on small screens
- tables/lists adapt to narrower viewports

Error Handling

implemented by: Armin Lachini

Related code-parts:

- `server/src/domains/commons/errors.ts`
 - defines a central `AppError` which contains central information for the error handler
 - the HTTP `statusCode`
 - the unique `appErrorCode` identifier
 - whether or not to `expose` the provided error message on the client
 - the error- `message` to display on the client
 - provides the middleware for express to map from the `AppError` to an HTTP-request and return it
- `client/src/commons/interceptors/serverError.interceptor.ts`
 - verifies the overall format of the response
 - logs out when the token is expired
 - catches the error message and displays it in a snackbar
- `server/src/middleware/not-found.ts`
 - the last route to match before the error handler
 - throws a `NotFoundError` handled by the error handler

Async Handler

implemented by: Armin Lachini

Related code-parts:

- `server/src/middleware/async-handler.ts`
 - wraps async route handlers to automatically catch errors
 - passes caught errors to the Express error middleware (see Error Handling)
 - eliminates the need for try-catch blocks in every async route

Navigation & Routing

implemented by: Armin Lachini

Related code-parts:

- `client/src/layout/navbar/*`
 - displays navigation links based on user roles
 - uses Angular signals for reactive state management
 - handles menu toggle and logout functionality

- `server/index.ts`
 - entrypoint for the Express application
 - mounts all route handlers and middleware

Distance Simulation

implemented by: Armin Lachini

Related code-parts:

- `server/src/domains/location/address.service.ts`
 - simulates delivery distance using a 21x21 grid (`grid_x` , `grid_y` from -10 to 10)
 - calculates Manhattan distance: `|x1 - x2| + |y1 - y2|`
 - estimates delivery time: 5 minutes per grid step
- `db/init-scripts/01-tables.sql`
 - stores `grid_x` and `grid_y` on addresses
 - stores `max_delivery_distance` on restaurants

Extra Tasks

implemented by: Miriam Manasreh (customer start-up & extra task details)

Real-Time Order Tracking

implemented by: Miriam Asreh

The order tracking component simulates WebSocket communication to provide customers with live order updates. This creates an engaging, real-time experience only by using WebSocket simulation.

Key features

1. WebSocket Simulation

- Connection simulation: simulates WebSocket handshake and connection duration
- Connection status: Visual indicator, indicating status when connected, ex (“Live”)
- Automatic reconnection: simulates reconnection attempts if connection disappears

2. Real-time state updates

- Order status automatically progresses based on specific timing intervals
- Dynamic ETA: delivery time updated based on current status and distance
- Order status changes:
 1. Placed -> Accepted
 2. Accepted -> Preparing
 3. Preparing -> Ready
 4. Ready -> Delivering
 5. Delivering -> Delivered

3. Live-Chat System

- Bi-directional Communication: Restaurant and customer send and receive messages
- Customer received messages by restaurant staff at each status update
- Customer sends messages to restaurant. Restaurant replies by default with “Thank you for the message. We will get back to you soon”
- Chat history is preserved throughout the order tracking duration
- Constant Popup Notifications: restaurant messages also appear as snackbar notification

4. Location-Based Tracking

- Grid Coordinate System: Users and Restaurants are assigned (x,y) coordinates
- Distance Calculation: Real-time distance updates as order progresses
- Visual Representation: Visual representation of order journey from order placement to delivery

5. Status Visualization

- Progress Bar: visual order journey representation
- Live updates: UI updates after each status update

Implementation Details

WebSocket Service Architecture:

```
// Core WebSocket simulation structure
class WebSocketService {
  private connectionStatus = new BehaviorSubject<boolean>(false);
  private messages = new Subject<ChatMessage>();
  private statusUpdates = new Subject<StatusUpdate>();

  connect(orderId: string, userId: string): void {
    // Simulate connection establishment
    setTimeout(() => {
      this.connectionStatus.next(true);
      this.simulateStatusUpdates(orderId);
      this.simulateRestaurantResponse(orderId);
    }, 1000);
  }

  simulateStatusUpdates(orderId: string): void {
    // Timeline-based status progression
    const timeline = [
      { delay: 30000, status: 'accepted' },
      { delay: 90000, status: 'preparing' },
      { delay: 300000, status: 'ready' },
      { delay: 1000, status: 'delivering' },
      { delay: this.calculateDeliveryTime(), status: 'delivered' },
    ];
  }
}
```

Chat Message Structure:

```
interface ChatMessage {
  id: string;
  senderId: string;
  senderName: string;
  senderType: 'customer' | 'restaurant';
  message: string;
  timestamp: Date;
  orderId: string;
  showPopup?: boolean; // For important notifications
}
```

Order Status Changes:

```
const statusTimeline = {
  placed: { duration: '0-30s', action: 'Order received' },
  accepted: { duration: '30s-2m', action: 'Restaurant confirmed' },
  preparing: { duration: '2-8m', action: 'Food preparation' },
  ready: { duration: '0-1m', action: 'Ready for delivery' },
  delivering: { duration: 'Distance-based', action: 'Out for delivery' },
  delivered: { duration: 'Final', action: 'Order completed' },
};
```

Challenges:

1. Real-time communication simulation - using Websocket **simulation** connection
2. Coordinate updates - ensuring status updates, chat messages and pop up messages are in sync
3. Visual representation

Key Features Summary

1. Restaurant Filtering: restaurant search and filtering
2. Cart Constraints: restaurant-specific cart
3. Order Tracking: real-time updates with visual representation
4. Payment Section: payment method options
5. Feedback Submission: post-delivery feedback submission
6. Real-Time Communication: Websocket chat simulation and status updates

Setup Instructions

Prerequisites:

- Node.js
- npm
- Angular CLI

Backend Setup

1. Navigate to server directory:
 - Type: `cd server`
2. Install dependencies:
 - Type: `npm install`
3. Build and start the server:
 - Type: `npm run build`
 - After type: `npm start`
4. If port 3000 is already in use, terminate the process using it:
 - Type: `netstat -ano | findstr :3000`
 - Then: `taskkill /PID [PID] /F`
5. Restart the server:
 - Type: `npm start`

Frontend Setup

1. Open new terminal and navigate to client directory:
 - Type: `cd client`
2. Install dependencies:
 - Type: `npm install`
3. Configure proxy for API calls - Make sure `proxy.conf.json` in client module has this code:

```
{
  "/api": {
    "target": "http://localhost:3000",
    "secure": false,
    "changeOrigin": true
  },
  "/uploads": {
    "target": "http://localhost:3000",
    "secure": false,
    "changeOrigin": true
  }
}
```

- **Important:** Docker usage was unverified due to persistent installation difficulties on the development laptop. This proxy configuration is a fallback solution.

4. Start Angular development server:

- Type: `ng serve`

5. Access application:

- Open browser and navigate to: `http://localhost:4200`
- Use the following test credentials:
 - Email: `customer@freelivery.com`
 - Password: `customer`

Environment Variables

No particular environment variables are required for local development. The application uses default configurations for:

- Database connection (local PostgreSQL)
- Server port (3000)
- API endpoints (localhost)

Setup Instructions

- open the project using the `freelivery.workspace` file
- install the recommended extensions
- run `npm i` in the project root
- set the environment variables (`sample.env`) as you need in a `.env` -file
- the complete development setup can be started using the `Full Stack: Debug Client + Server` task in VS-Code
 - if you're in a different IDE you can start the project using `docker compose up -d`
 - DB-migrations run automatically on the first startup, or whenever the volume is pruned using `docker compose down -v`